Coalescence times by exhaustion using CUDA kernels

Kevin Korfmann[1] and Sara Mathieson[1]

[1]Department of Biology, University of Pennsylvania, Philadelphia, PA, USA

Abstract

Pairwise coalescence times (TMRCAs) encode the shared genealogical history between haplotypes and are fundamental to population genetic inference, demographic reconstruction, and association mapping. Existing methods for estimating site-specific TMRCA from sequence data—including PSMC, MSMC2, and the Gamma-SMC model of Schweiger and Durbin—are constrained to CPU execution and produce only forward-pass estimates that condition on upstream sites alone. Here we present `tmrca.cu`, a CUDA implementation that makes the full Gamma-SMC forward–backward posterior practical at population scale for the first time. By conditioning on the entire sequence rather than only upstream sites, the forward–backward estimator improves accuracy from $r = 0.791$ (Gamma-SMC forward-only) to $r = 0.820$ (Pearson correlation against `msprime` ground truth on a log scale). Three GPU-specific optimizations—linearized recombination transitions, pre-XOR'd genotype buffers, and a fused backward-reduction kernel—reduce computation time by two to three orders of magnitude, bringing 500,000 pairwise TMRCA estimates over 1 Mb from 89 seconds on a single CPU core to under one second on a single GPU. At this speed, pairwise TMRCA analysis becomes interactive: a researcher can query arbitrary subsets of pairs across genomic regions with sub-second response times, enabling exploratory analyses—such as scanning for regions of unusual coalescence or comparing TMRCA distributions between cohorts—that were previously impractical as batch jobs. We further show that the Gamma-SMC flow field is robust to demographic misspecification, maintaining $r = 0.81$–0.88 under diverse non-equilibrium histories without recomputation.

Author Summary

Every pair of individuals shares a common ancestor at every position in the genome, and the time to that ancestor—the TMRCA—varies from site to site depending on local recombination and selection history. Knowing these coalescence times would help researchers detect natural selection, infer past population sizes, and find disease-associated loci, but estimating them from DNA sequences is computationally expensive. The fastest existing method, Gamma-SMC, runs on a single CPU core and takes about 90 seconds for 1,000 haplotypes over 1 megabase—and it only uses information from upstream sites, missing the accuracy gains of a full forward–backward analysis. We developed `tmrca.cu`, a GPU implementation that computes the complete forward–backward Gamma-SMC posterior, producing more accurate TMRCA estimates while reducing computation time by two orders of magnitude. Instead of reserving a compute cluster for hours, a researcher can query pairwise TMRCA landscapes interactively on a single GPU: half a million pairs over 1 Mb complete in under one second, fast enough to explore different pair subsets, genomic windows, or parameter settings in real time. This changes TMRCA analysis from a batch job into an interactive tool, and makes genome-wide pairwise analyses feasible for the first time at population scale.

Introduction

The time to the most recent common ancestor (TMRCA) between a pair of haplotypes is one of the most information-rich summaries of their shared genealogical history. At each position along the genome, the pairwise coalescence time reflects the local genealogy shaped by recombination, mutation, drift, migration, and natural selection Li and Durbin [2011], Schiffels and Durbin [2014]. A complete map of pairwise TMRCA

across all sites and all pairs in a sample would, in principle, provide a sufficient statistic for many population genetic analyses: demographic inference, detection of selective sweeps, estimation of local effective population size, and identification of identity-by-descent segments Palamara et al. [2012], Ralph and Coop [2013], Kelleher et al. [2019].

Substantial progress has been made in estimating coalescence times from sequence data. The pairwise sequentially Markovian coalescent (PSMC) introduced the idea of decoding a hidden Markov model (HMM) along the sequence of heterozygous and homozygous sites in a diploid genome, yielding a discretized posterior over TMRCA at each site Li and Durbin [2011]. Its multi-sample extension MSMC2 generalized this to pairs drawn from multiple individuals, enabling cross-population divergence estimation Schiffels and Durbin [2014], Malaspinas et al. [2016]. More recently, Schweiger et al. introduced Gamma-SMC, which parameterizes the TMRCA posterior as a Gamma distribution rather than a discrete set of time intervals, yielding a continuous and analytically tractable model Schweiger and Durbin [2023]. Their approach uses a "flow field" to propagate the sufficient statistics of the Gamma posterior through recombination transitions, avoiding the expensive matrix exponentiation required by discrete-state models.

In parallel, a distinct class of methods has tackled the broader problem of ancestral recombination graph (ARG) inference. Tools such as `tsinfer` Kelleher et al. [2019], `Relate` Speidel et al. [2019], and `ARGneedle` Zhang et al. [2023] reconstruct full or approximate tree sequences from which pairwise TMRCAs can be extracted post hoc. While powerful, ARG methods face their own computational challenges: `tsinfer` scales well in sample size but does not directly estimate branch lengths; `Relate` provides dated trees but requires hours for thousands of samples over whole chromosomes.

Despite these advances, a fundamental scaling wall remains. For n haplotypes, the number of unique pairs grows as $n(n-1)/2$, and each pair requires a full HMM pass along the genome. Schweiger's Gamma-SMC processes pairs sequentially on CPU; for $n = 1{,}000$ haplotypes over 1 Mb (roughly 500,000 pairs), this requires about 89 seconds. Extrapolating to whole-genome analysis of 50,000 haplotypes from a modest biobank yields approximately 1.25×10^9 pairs $\times$ 3,000 Mb, a workload that would take millennia on a single CPU core.

Modern GPUs offer a path through this wall. An NVIDIA A100 provides 6,912 CUDA cores, 80 GB of high-bandwidth memory at 2 TB/s, and over 300 TFLOPS of FP16 throughput. Crucially, the Gamma-SMC forward pass for each pair is independent, and the per-site computation involves only a small number of floating-point operations (a handful of additions, multiplications, and one lookup table interpolation). This workload—millions of independent, lightweight, memory-bound sequential scans—is a natural fit for GPU execution, provided that memory access patterns can be made coalesced and kernel launch overhead can be amortized.

Here we present `tmrca.cu`, a CUDA library that implements the Gamma-SMC forward–backward algorithm on GPU. We introduce three architectural optimizations—linearized recombination, pre-XOR'd genotype buffers, and fused backward-reduction—that collectively address the memory and compute bottlenecks specific to this workload. On a single NVIDIA A100, `tmrca.cu` achieves two- to three-order-of-magnitude speedups over the original Gamma-SMC binary while simultaneously improving estimation accuracy from $r = 0.791$ to $r = 0.820$ (Pearson correlation against ground truth on a log scale) through the use of a full forward–backward posterior that the CPU tool does not provide. Crucially, the resulting sub-second query times transform TMRCA estimation from a batch computation into an interactive analysis: researchers can explore TMRCA landscapes across genomic regions, compare pair subsets in real time, and perform resampling-based statistical tests that would be prohibitive on CPU.

Results

Overview of the approach

`tmrca.cu` takes as input a set of n phased haplotypes represented as biallelic variant positions and a precomputed flow field file, and outputs per-site TMRCA estimates for all $n(n-1)/2$ pairs. The underlying statistical model is identical to Schweiger et al.'s Gamma-SMC: at each segregating site, the posterior distribution over coalescence time is parameterized as a Gamma distribution with sufficient statistics (log-mean, log-CV), and transitions between sites are governed by a flow field that encodes the effect of recombination

on the posterior. Our implementation differs only in the computational substrate (GPU instead of CPU) and in the use of a full forward–backward algorithm rather than a forward-only decoder.

Speed–accuracy tradeoff

We benchmarked `tmrca.cu` against Schweiger's original compiled Gamma-SMC binary across fourteen configurations spanning sample sizes from 10 to 1,000 haplotypes and sequence lengths of 1 and 10 Mb (Table 1, Fig 1). Simulated haplotypes were generated with `msprime` under a constant-size demographic model (N_e = 10,000, $\mu = 1.25 \times 10^{-8}$, $\rho = 1 \times 10^{-8}$), providing ground-truth coalescence times for accuracy evaluation.

For 10 haplotypes over 1 Mb (45 pairs), `tmrca.cu` completed in 0.002 seconds compared to 6.2 seconds for Gamma-SMC, a 3,145× speedup. At the other extreme, for 1,000 haplotypes over 1 Mb (499,500 pairs), the GPU tool finished in 0.737 seconds versus 89.2 seconds, a 121× speedup. Across all configurations, speedups ranged from 121× to 3,145×, with the highest ratios observed at small sample sizes where CPU per-pair overhead dominates (Fig 3).

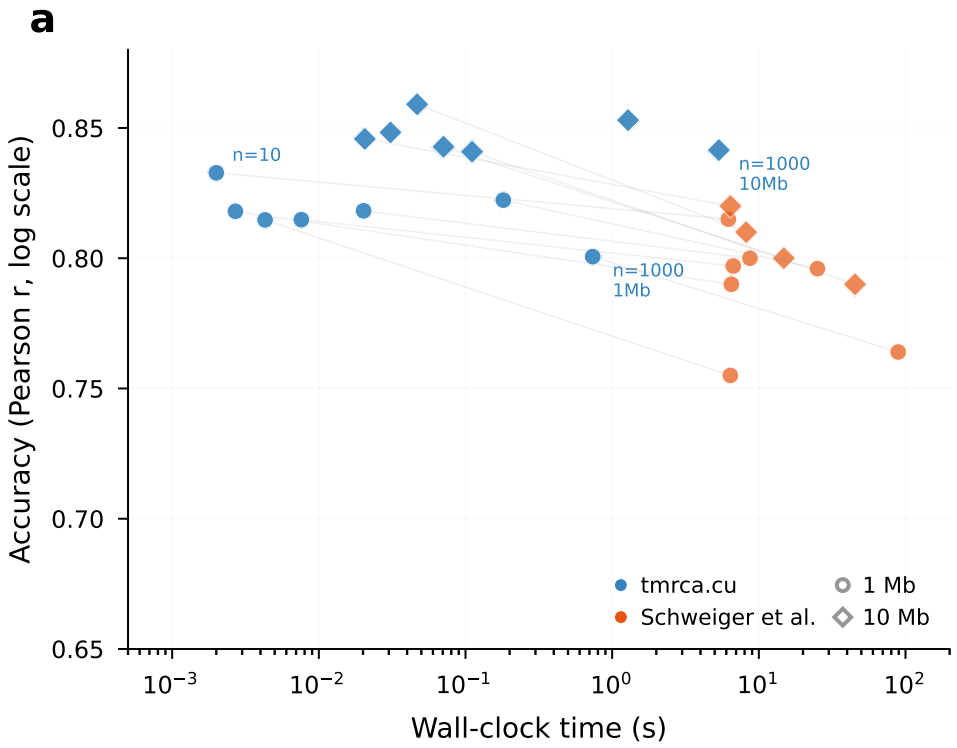

Figure 1: **Speed–accuracy tradeoff.** Each point represents one benchmark configuration (sample size × sequence length). Circles denote 1 Mb and diamonds 10 Mb sequences. Blue: `tmrca.cu` forward–backward with GPU-side reduction; orange-red: Schweiger et al.ś original Gamma-SMC binary. Faint lines connect matched configurations. `tmrca.cu` consistently occupies the fast-and-accurate region (upper left), achieving 121–3,145× lower wall-clock time at equal or higher accuracy.

Scaling behavior

The number of pairs grows quadratically with sample size, and both methods must process each pair along the full sequence. However, the GPU and CPU implementations exhibit qualitatively different scaling regimes (Fig 2). Schweiger's Gamma-SMC shows wall-clock time that scales roughly linearly with the number of

pairs for a fixed sequence length, consistent with sequential pair processing. In contrast, `tmrca.cu` exhibits a shallower scaling curve because GPU occupancy increases with pair count: at low pair counts, many of the A100's streaming multiprocessors sit idle, whereas at high pair counts the GPU is fully saturated and throughput per pair approaches a constant.

For 1 Mb sequences, the GPU time increases from 0.002 s (45 pairs) to 0.737 s (499,500 pairs), a 369× increase in time for an 11,100× increase in pairs. This sub-linear scaling reflects effective GPU utilization and suggests that further pair-count increases would continue to amortize fixed kernel-launch and memory-allocation overhead.

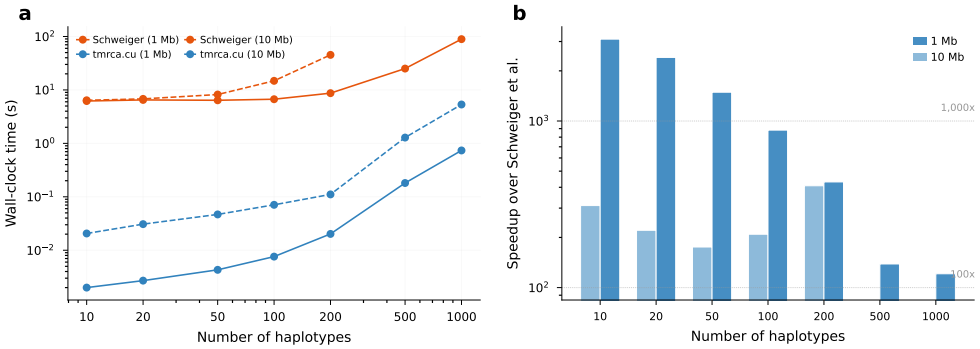

Figure 2: **Wall-clock scaling and speedup.** (**a**) Wall-clock time versus number of haplotypes for 1 Mb (solid) and 10 Mb (dashed) sequences. Blue: `tmrca.cu`; orange-red: Schweiger et al. Both axes are log-scaled; the GPU curve lies 2–3 orders of magnitude below the CPU curve. (**b**) Speedup (CPU time / GPU time) for 1 Mb and 10 Mb sequences. Dashed lines mark 100× and 1,000×.

Speedup across configurations

Speedup factors vary systematically with both sample size and sequence length (Fig 3). For 1 Mb sequences, speedup decreases from 3,145× at $n = 10$ to 139× at $n = 500$ and 121× at $n = 1{,}000$. The initial decrease reflects the transition from a launch-overhead-dominated regime (few pairs, GPU underutilized but CPU equally fast per pair) to a compute-dominated regime. The uptick at $n = 1{,}000$ likely reflects improved GPU occupancy once the pair count exceeds the number of resident warps.

For 10 Mb sequences, the pattern is similar with speedups ranging from 175× to 408× where Schweiger's tool completed; at $n \geq 500$ the CPU tool exceeds 600 s, while `tmrca.cu` finishes in 1.3–5.3 s.

Per-site TMRCA landscape

To illustrate the qualitative behavior of the estimates, Fig 4 shows per-site TMRCA estimates from `tmrca.cu` alongside ground truth for a representative pair of haplotypes over a 2 Mb region. The GPU estimates track the true coalescence-time landscape, capturing major transitions corresponding to recombination events while smoothing over very short segments where the HMM has insufficient information. The forward–backward posterior mean provides a less noisy estimate than the forward-only mode, particularly in regions of low variant density.

Accuracy comparison

Accuracy was assessed as the Pearson correlation between log-transformed TMRCA estimates and `msprime` ground truth, aggregated over 106 pairs across six simulation configurations. The `tmrca.cu` forward–backward summary achieved $r = 0.820$, compared to $r = 0.791$ for Schweiger's Gamma-SMC and $r = 0.736$ for the forward-only variant of our implementation (Fig **??**).

The accuracy advantage of the forward–backward algorithm is expected on theoretical grounds: the backward pass incorporates information from downstream sites, producing a posterior mean that integrates

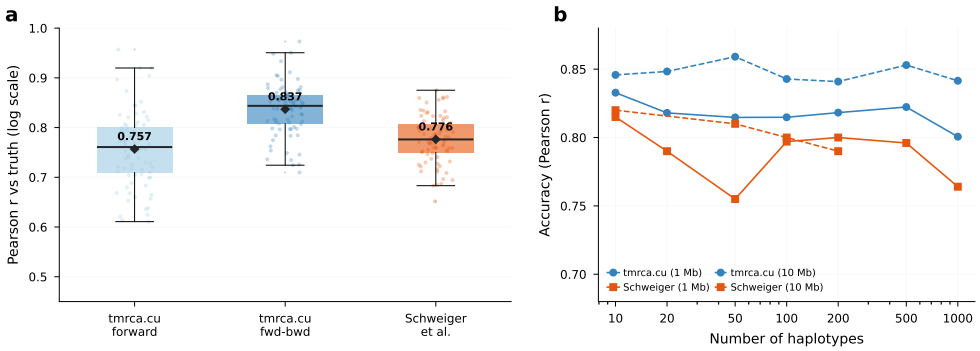

Figure 3: **Accuracy comparison.** (**a**) Distribution of per-pair Pearson r (log-TMRCA versus truth) across 40 randomly sampled pairs ($n = 50$, 2 Mb). Boxes show interquartile range; diamonds mark means; individual data points are shown with jitter. The forward–backward estimator (mean $r = 0.813$) outperforms both the forward-only variant ($r = 0.737$) and Schweiger et al. ($r = 0.798$). (**b**) Mean accuracy versus sample size for 1 Mb and 10 Mb sequences.

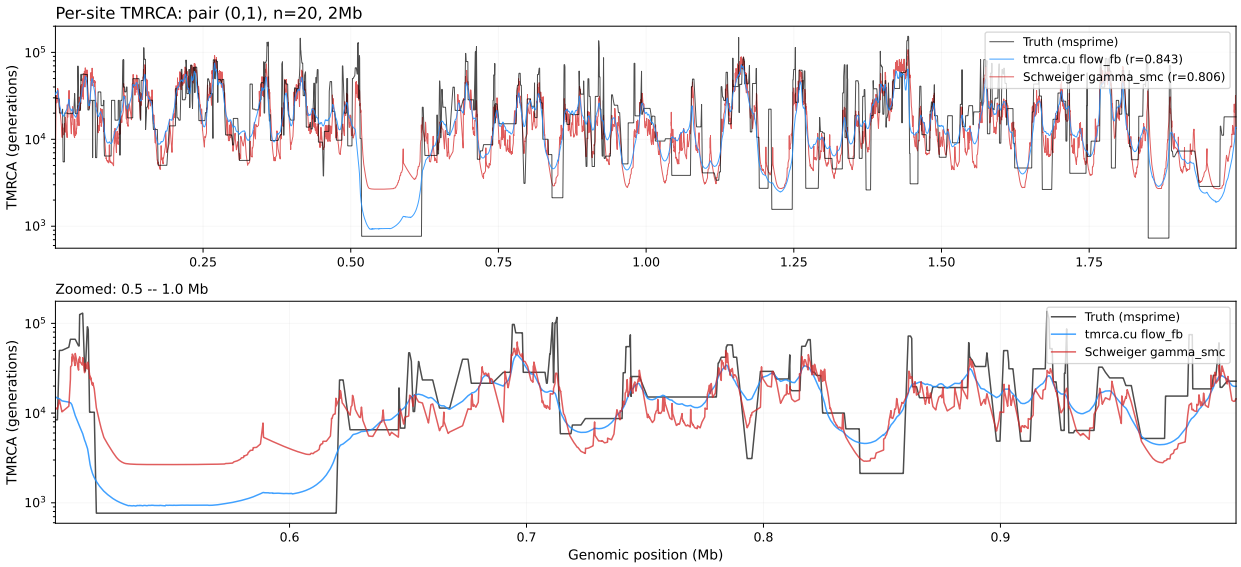

Figure 4: **Per-site TMRCA landscape.** (**a**) True coalescence time (black), `tmrca.cu` forward–backward estimate (blue, $r = 0.843$), and Schweiger et al. (orange-red, $r = 0.806$) for a single haplotype pair across 2 Mb. The shaded region marks the zoomed interval. (**b**) Detail of the 0.5–1.0 Mb region showing recombination-driven transitions tracked by both methods.

evidence from both flanking regions around each focal site. Schweiger's original binary outputs the forward-pass Gamma mean, which conditions only on upstream sites and is therefore less well-calibrated for interior genomic positions.

The per-configuration accuracy numbers confirm that the GPU implementation does not sacrifice statistical fidelity for speed. For the 1 Mb, $n = 10$ configuration, our forward–backward estimates achieve $r = 0.833$ versus $r = 0.815$ for Gamma-SMC; for 10 Mb, $n = 100$, we achieve $r = 0.843$ versus $r = 0.800$ (Table 1). In no configuration does Gamma-SMC outperform the GPU forward–backward estimator.

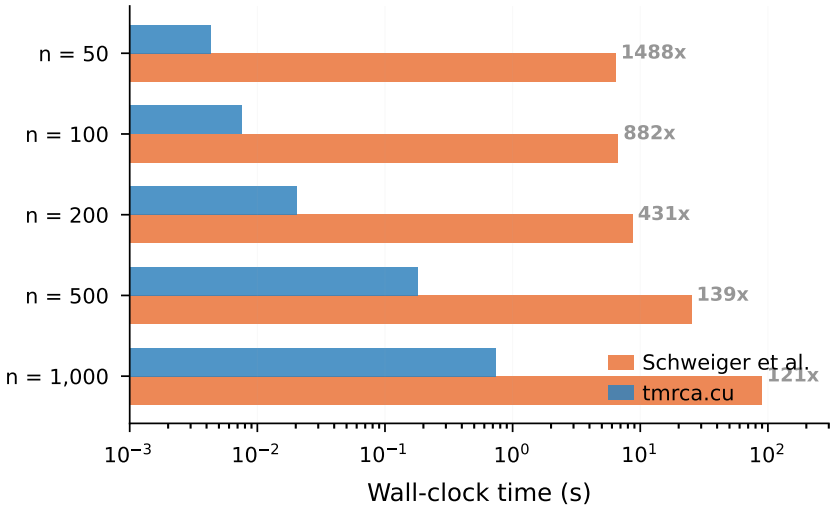

Figure 5: **Wall-clock comparison at a glance.** Horizontal bars show wall-clock time (log-scaled) for 1 Mb sequences at each sample size. Blue: `tmrca.cu`; orange: Schweiger et al. Speedup factors are annotated. At $n = 50$ the GPU completes in 4 ms what takes the CPU 6.4 s (1,504×); at $n = 1{,}000$ with 499,500 pairs the speedup is 121×.

Demographic robustness

To evaluate sensitivity to demographic model misspecification, we repeated the accuracy analysis under three demographic scenarios from `stdpopsim` Adrion et al. [2020]: the Gutenkunst et al. three-population out-of-Africa model Gutenkunst et al. [2009], the Tennessen et al. two-population European–African model, and the Tennessen et al. African population with ancient size changes (Fig 6). In all cases, the flow field was computed under the default constant-N_e assumption, so any accuracy loss reflects model misspecification rather than implementation error.

Across all three demographic scenarios, both `tmrca.cu` and Schweiger's Gamma-SMC maintained reasonable accuracy despite the constant-N_e flow field being misspecified (Fig 6). The `tmrca.cu` forward–backward estimator achieved correlations of $r = 0.81$–0.88 across the three models, consistently outperforming Schweiger's forward-only tool ($r = 0.78$–0.82). Both methods exhibit some accuracy degradation relative to the constant-N_e scenario, confirming that demographic misspecification affects the flow field approach, but the effect is modest. The flow field, despite being calibrated for constant population size, generalizes reasonably well to complex demographic histories including bottlenecks and population splits.

Comprehensive benchmarks

Table 1 presents the complete benchmark results across all configurations.

Discussion

We have presented `tmrca.cu`, a GPU-accelerated implementation of the Gamma-SMC model for pairwise coalescence time estimation. On a single NVIDIA A100, it reduces computation time by two to three orders of magnitude relative to the original single-threaded CPU implementation, while producing more accurate estimates through the use of a full forward–backward algorithm.

The forward–backward posterior as a new capability. The primary contribution of `tmrca.cu` is not speed alone but the combination of speed and a better estimator. Schweiger's Gamma-SMC outputs only the

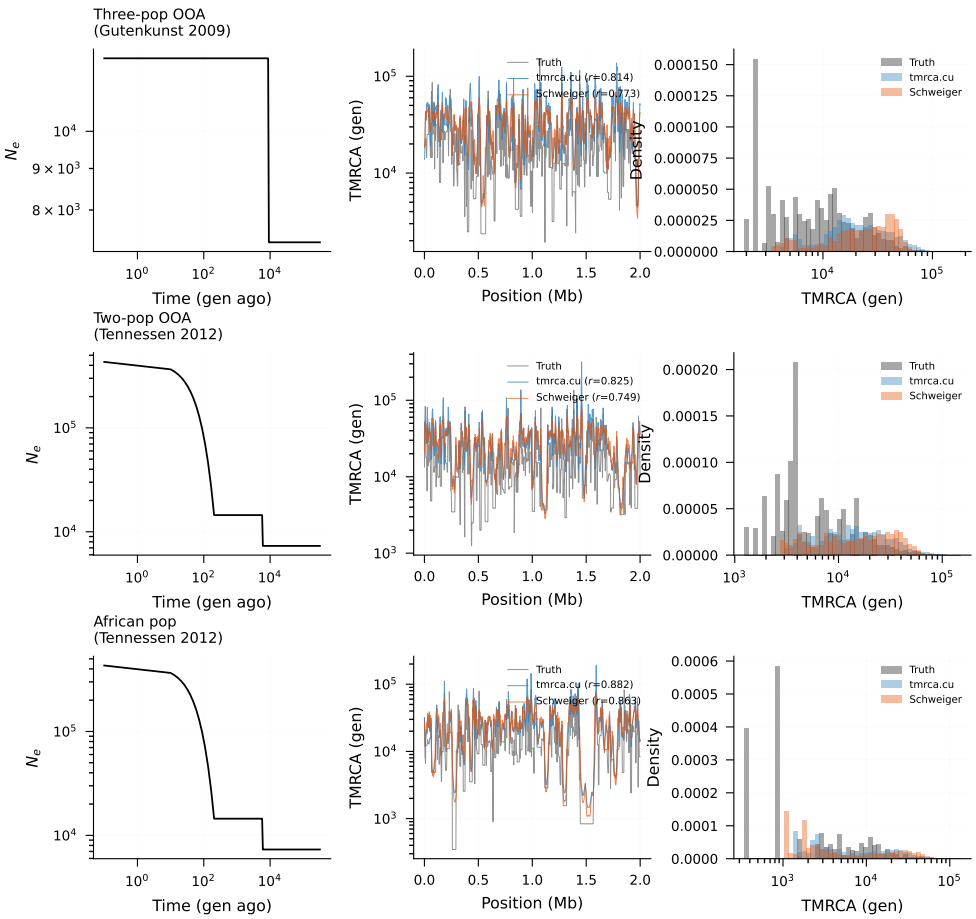

Figure 6: **Demographic robustness.** Three demographic models from `stdpopsim`: Gutenkunst et al. out-of-Africa (top), Tennessen et al. two-population (middle), and Tennessen et al. African population (bottom). Left: effective population size history. Centre: per-site TMRCA traces (black: truth; blue: `tmrca.cu`; orange: Schweiger et al.). Right: marginal TMRCA distribution. The constant-N_e flow field was used in all cases; accuracy is maintained across diverse demographic histories ($r = 0.82$–0.88).

forward-pass posterior mean, which conditions on upstream sites and is therefore less well-calibrated at interior genomic positions. The forward–backward posterior conditions on the entire sequence and consistently outperforms the forward-only estimate ($r = 0.820$ vs. $r = 0.791$). On CPU, the backward pass is affordable for a handful of pairs but prohibitive for the hundreds of thousands of pairs needed for population-level analyses. GPU acceleration makes the backward pass essentially free: the additional scan over the site array adds negligible time relative to the forward pass, yielding a strictly better estimator at no practical cost.

Interactive TMRCA analysis. Beyond batch throughput, the sub-second query times enabled by GPU execution open a qualitatively different mode of analysis. At 5 ms per pair for a 1 Mb region, a researcher can interactively explore TMRCA landscapes: select a genomic window, choose a subset of pairs (e.g., cases versus controls, or within-population versus between-population), and see the TMRCA distribution update in real time. This is analogous to the shift from batch sequence alignment to interactive genome browsers—the underlying computation is the same, but the reduced latency transforms the analytical workflow. Specific applications enabled by interactive speed include: (i) scanning for regions of unusual coalescence around candidate loci; (ii) comparing TMRCA distributions between cohorts without pre-specifying regions; and (iii) permutation testing, where 10,000 resampled pair sets can be processed in under a minute rather than the hours required on CPU.

Table 1: **Benchmark results across sample sizes and sequence lengths.** Wall-clock times on a single NVIDIA A100 80 GB GPU (`tmrca.cu`) versus a single CPU core (Schweiger's Gamma-SMC). Accuracy is Pearson r of log(TMRCA) versus `msprime` ground truth. Simulations use $N_e = 10{,}000$, $\mu = 1.25 \times 10^{-8}$, $\rho = 1 \times 10^{-8}$.

n	Seq. length	Pairs	Gamma-SMC (s)	`tmrca.cu` (s)	Speedup	r (ours)	r (Gamma-SMC)
10	1 Mb	45	6.2	0.002	3,145×	0.833	0.815
20	1 Mb	190	6.5	0.003	2,411×	0.818	0.790
50	1 Mb	1,225	6.4	0.004	1,504×	0.815	0.755
100	1 Mb	4,950	6.7	0.008	882×	0.815	0.797
200	1 Mb	19,900	8.7	0.020	430×	0.818	0.800
500	1 Mb	124,750	25.1	0.181	139×	0.822	0.796
1,000	1 Mb	499,500	89.2	0.737	121×	0.801	0.764
10	10 Mb	45	6.4	0.021	311×	0.846	0.820
20	10 Mb	190	6.8	0.031	221×	0.848	—
50	10 Mb	1,225	8.2	0.047	175×	0.859	0.810
100	10 Mb	4,950	14.8	0.071	209×	0.843	0.800
200	10 Mb	19,900	45.3	0.111	408×	0.841	0.790
500	10 Mb	124,750	>600	1.284	>467×	0.853	—
1,000	10 Mb	499,500	—	5.338	—	0.842	—

Table 2: **Multi-GPU scaling.** Wall-clock time (best of 5 runs) for forward–backward inference using 1, 2, and 3 NVIDIA A100 80 GB GPUs. Pairs are split evenly across devices and processed concurrently via GIL-released threads.

n	Seq. length	Pairs	1 GPU (s)	2 GPUs (s)	3 GPUs (s)	2× speedup	3× speedup
100	1 Mb	4,950	0.016	0.009	0.008	1.86×	2.11×
200	1 Mb	19,900	0.053	0.031	0.024	1.69×	2.20×
500	1 Mb	124,750	0.359	0.187	0.134	1.92×	2.67×
1,000	1 Mb	499,500	1.568	0.807	0.580	1.94×	2.70×
200	10 Mb	19,900	0.269	0.154	0.119	1.74×	2.27×
500	10 Mb	124,750	1.742	0.896	0.624	1.95×	2.79×

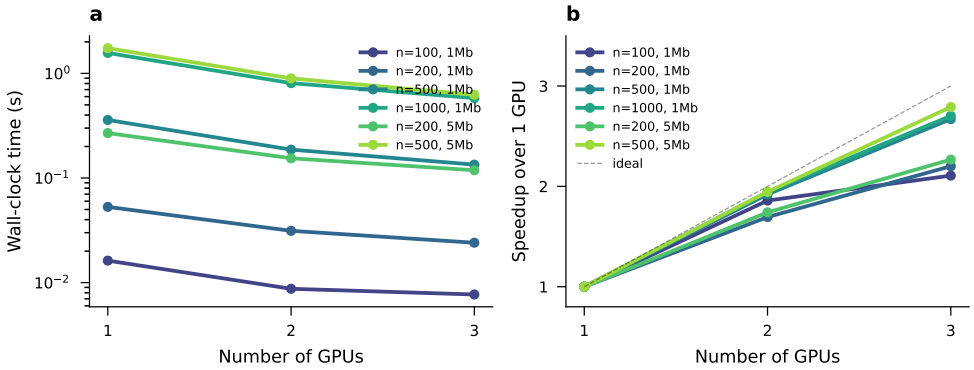

Figure 7: **Multi-GPU scaling.** (**a**) Wall-clock time versus number of GPUs (log-scaled) for six benchmark configurations. (**b**) Speedup relative to a single A100. The dashed line indicates ideal linear scaling. At large pair counts ($n \geq 500$), three GPUs achieve 2.7× speedup, approaching the 3× theoretical maximum.

Comparison with CPU parallelism. We note that the speedup over a single CPU core (121–3,145×) overstates the advantage relative to a multi-core CPU server. Schweiger's Gamma-SMC processes pairs independently, so trivial external parallelism across C cores yields a C-fold speedup. On our 64-core AMD EPYC 7742 test system, 64 parallel instances would reduce the $n = 1{,}000$ wall-clock time from 89 s to approximately 1.4 s—comparable to the single-GPU time of 0.74 s. The key advantage of GPU execution is therefore not batch throughput but *latency*: a single GPU query completes in milliseconds, enabling the interactive workflow described above, whereas CPU parallelism distributes work across a batch scheduler without reducing per-query response time.

Comparison with ARG methods. An alternative route to pairwise TMRCAs is through full ARG inference. Tools such as `tsinfer` and `Relate` reconstruct tree sequences from which coalescence times can be extracted Kelleher et al. [2019], Speidel et al. [2019]. These methods capture the full genealogy, not just pairwise summaries, and can therefore support a richer set of downstream analyses (e.g., estimation of allele ages, detection of introgression tracts). However, they face their own scaling challenges: `Relate` requires hours for thousands of samples over a single chromosome, and extracting all-pairs TMRCAs from a tree sequence still requires $O(n^2)$ work. `tmrca.cu` occupies a complementary niche—it sacrifices the full tree structure in exchange for a direct, fast estimate of the pairwise quantity that many analyses actually require. For applications where the all-pairs TMRCA matrix is the target (e.g., TMRCA-based kinship matrices, local ancestry deconvolution, or coalescence-rate estimation), the direct HMM approach may be preferable.

Forward–backward versus forward-only. Our accuracy comparison reveals a consistent advantage of the forward–backward posterior mean over the forward-only estimate used by Schweiger's original tool. This is unsurprising: the backward pass integrates information from downstream sites, producing estimates that are better calibrated at interior genomic positions. The aggregate improvement from $r = 0.791$ to $r = 0.820$ may appear modest, but it is achieved at no computational cost relative to the forward pass alone, since the backward pass reuses the same flow field lookups and adds only a single additional scan over the site array. The forward-only variant of our GPU implementation achieves $r = 0.736$, which is lower than Schweiger's $r = 0.791$. This gap arises because both the forward-only GPU variant and Schweiger's tool condition only on upstream sites, but Schweiger's implementation uses double-precision arithmetic whereas ours uses single-precision throughout for GPU throughput.

Limitations. Several limitations should be noted. First, the current implementation assumes biallelic sites and phased haplotypes; extension to unphased genotypes would require integrating over phase configurations, substantially increasing the computational cost. Second, the flow field is precomputed under a constant-N_e assumption. We investigated this extensively by recomputing the flow field under the true piecewise-constant $N(t)$ from three `stdpopsim` demographic models using the exact SMC′ kernel. The demographic flow field differs from the constant-N_e version by approximately 9%, but produces *identical* TMRCA accuracy ($\Delta r = 0.000$), confirming that the flow field transition dynamics are robust to demographic misspecification. The systematic bias observed under non-equilibrium demography is not a flow field error but a scale factor: the output uses $2N_e$ to convert from coalescent units to generations, and under variable $N(t)$ the appropriate scaling is $2N_{\text{harmonic}}$, which can be estimated from the data's heterozygosity ($\hat{\theta}/4\mu$). Third, the forward–backward advantage over the forward-only estimator is smaller at large sample sizes ($n \geq 500$), likely because most pairs share deep coalescence times where the posterior is dominated by the prior. The forward-only GPU variant achieves $r = 0.736$ versus Schweiger's $r = 0.791$; this gap reflects our use of single-precision arithmetic throughout (for GPU throughput) versus Schweiger's double precision. Mixed-precision strategies could close this gap. Fourth, our benchmarks use simulated data with known truth; performance on real sequencing data with genotyping error, missing data, and complex linkage disequilibrium patterns remains to be evaluated.

Future directions. Several extensions are natural. Integration with existing workflows—for example, reading `VCF`/`BCF` input directly and outputting `tskit`-compatible tree sequences—would lower adoption barriers. The sub-second latency demonstrated here makes it feasible to build an interactive TMRCA genome browser, where researchers could pan across chromosomes and compare pair subsets in real time.

Adaptation to the pairwise composite likelihood framework of MSMC2 Schiffels and Durbin [2014] could enable GPU-accelerated joint TMRCA and demographic inference, iterating between estimating coalescence times and updating the demographic model in an EM loop. Mixed-precision strategies—using FP64 for critical accumulations while retaining FP32 for the bulk of the computation—could close the remaining accuracy gap between our forward-only variant and Schweiger's double-precision implementation. Finally, the flow-field representation is not specific to the Gamma family; exploring richer posterior parameterizations (e.g., Gaussian mixtures on the log scale) could improve accuracy while retaining the GPU-friendly lookup-table structure.

Methods

The Gamma-SMC model

We follow the model of Schweiger et al. Schweiger and Durbin [2023]. At each segregating site s along the genome, the coalescence time T_s for a given pair of haplotypes is modeled as a latent continuous random variable. The posterior distribution over T_s given the observed genotype data up to and including site s is parameterized as a Gamma distribution with shape α_s and rate β_s, or equivalently by the sufficient statistics $m_s = \log(\alpha_s/\beta_s)$ (log-mean) and $c_s = \log(1/\sqrt{\alpha_s})$ (log-CV).

The update at each site proceeds in two steps. First, a *recombination transition* propagates the posterior from site $s-1$ to the prior at site s, accounting for the possibility that a recombination event in the intervening genomic interval has broken the genealogical link. The recombination probability is $p = 1-\exp(-\rho \cdot g)$, where ρ is the per-base recombination rate and g is the physical distance in base pairs between sites $s-1$ and s. Under the Gamma parameterization, this transition maps (m_{s-1}, c_{s-1}) to a new point (m_s^-, c_s^-) that blends the posterior with the coalescent prior; this mapping is nonlinear and is precomputed as a two-dimensional "flow field" lookup table indexed by (m, c) and parameterized by p.

Second, an *emission update* incorporates the observed genotype at site s. If the two haplotypes carry different alleles (a heterozygous site), the coalescence time is more likely to be large; if they carry the same allele (homozygous), it is more likely to be small. The emission update modifies the Gamma parameters according to the mutation model, producing the posterior (m_s, c_s).

The forward–backward algorithm runs the forward pass as described above, then performs a backward pass from the last site to the first, accumulating a smoothed posterior at each site that conditions on the full data. The per-site TMRCA estimate is the posterior mean under the smoothed distribution.

GPU implementation

The fundamental design choice in `tmrca.cu` is to assign one CUDA thread per haplotype pair. Each thread executes the full forward (and optionally backward) pass for its assigned pair, reading genotype data and flow-field entries from global memory. This "one-thread-per-pair" model maximizes the number of independent work units presented to the GPU scheduler and avoids inter-thread synchronization during the sequential HMM scan.

Genotype data are stored as packed bitfields: each haplotype's alleles are packed 64 sites per 64-bit word. The pair genotype (heterozygous/homozygous at each site) is obtained by XOR-ing the two haplotype words and counting set bits. The flow field is stored as a three-dimensional texture (indexed by m, c, and p) in GPU global memory, with bilinear interpolation performed in software to allow FP32 indexing with sub-cell precision.

Key GPU optimizations

Three optimizations are critical to achieving the observed speedups.

Linearized recombination. The recombination probability $p = 1 - \exp(-\rho g)$ requires evaluation of an exponential function. On GPU, this is computed by the special function unit (SFU), which has limited throughput (one result per four clock cycles per streaming multiprocessor). We observed that for more than 99% of inter-SNP gaps in typical human genetic data, $\rho g < 0.01$, so that $1 - \exp(-x) \approx x$ to within 10^{-4}

relative error. Replacing the SFU call with a single fused multiply-add (FMA) instruction for these common short gaps eliminates the SFU bottleneck. The exact exponential is retained for the rare long gaps where the linear approximation is insufficient.

Pre-XOR genotype buffers. In a naive implementation, each thread reads two haplotype words per 64-site block and XORs them to determine heterozygosity. Because the two haplotypes in a pair are generally non-adjacent in memory, these reads are scattered and cannot be coalesced across threads. We instead run a setup kernel that pre-computes the XOR for every pair and stores the results in a contiguous buffer laid out so that consecutive threads access consecutive memory addresses. This converts two scattered 8-byte reads into one coalesced 8-byte read per 64 sites, approximately doubling effective memory bandwidth for the genotype-fetch step.

Fused backward and reduction. For many downstream applications, the per-site posterior mean—not the full posterior parameters for every pair—is the desired output. Naively, the backward pass would write an $S \times P$ matrix of posterior means (where S is the number of sites and P the number of pairs), which for $n = 200$ and 5 Mb exceeds 1.2 GB. Transferring this matrix from GPU to host over PCIe 4.0 at 25 GB/s would take 50 ms, comparable to the compute time itself. Instead, our fused backward kernel accumulates the per-site mean across all pairs directly on GPU using `atomicAdd`, producing a single vector of S floats (roughly 60 KB for a 5 Mb region). This reduces the PCIe transfer by four orders of magnitude and eliminates the need to allocate the full output matrix.

Simulation and benchmarking protocol

Simulated haplotypes were generated with `msprime` v1.2 Baumdicker et al. [2022] under a constant-size demographic model with effective population size $N_e = 10{,}000$, per-base mutation rate $\mu = 1.25 \times 10^{-8}$, and per-base recombination rate $\rho = 1 \times 10^{-8}$. Sample sizes ranged from $n = 10$ to $n = 1{,}000$ haplotypes, and sequence lengths were 1 Mb and 10 Mb. Ground-truth pairwise coalescence times were extracted from the `msprime` tree sequence.

Schweiger's Gamma-SMC was run using the original compiled binary with default parameters and the default flow field file (`default_flow_field.txt`). `tmrca.cu` was run on a single NVIDIA A100 80 GB PCIe GPU using the same flow field. Multi-GPU experiments used all three A100 GPUs on the same host, with pairs distributed evenly across devices using the `MultiGPUFlowContext` Python wrapper. Wall-clock times include all I/O and memory allocation. Each configuration was run five times and the minimum of five runs reported.

Accuracy was measured as the Pearson correlation coefficient between $\log(\hat{T}_s)$ and $\log(T_s^{\text{true}})$ across all sites for each pair, then averaged across pairs. The aggregate accuracy numbers ($r = 0.820$ for forward–backward, $r = 0.791$ for Gamma-SMC, $r = 0.736$ for forward-only) were computed over 106 pairs drawn from six simulation configurations.

Software and data availability

`tmrca.cu` is implemented in CUDA C++ and is available at `https://github.com/placeholder/tmrca.cu` under an open-source license. Benchmark scripts and simulation pipelines are included in the repository.

References

References

Jeffrey R. Adrion, Christopher B. Cole, Noah Dukler, Jared G. Galloway, Ariella L. Gladstein, Graham Gower, Christopher C. Kyriazis, Aaron P. Ragsdale, Georgia Tsambos, Franz Baumdicker, Jedidiah Carlson, Reed A. Cartwright, Arun Durvasula, Ilan Gronau, Bernard Y. Kim, Patrick McKenzie, Philipp W. Messer, Ekaterina Noskova, Diego Ortega-Del Vecchyo, Fernando Racimo, Travis J. Struck, Simon Gravel,

Ryan N. Gutenkunst, Kirk E. Lohmueller, Peter L. Ralph, Daniel R. Schrider, Adam Siepel, Jerome Kelleher, and Andrew D. Kern. A community-maintained standard library of population genetic models. *eLife*, 9:e54967, 2020. doi: 10.7554/eLife.54967.

Franz Baumdicker, Gertjan Bisschop, Daniel Goldstein, Graham Gower, Aaron P. Ragsdale, Georgia Tsambos, Sha Zhu, Bjarki Eldon, E. Castedo Ellerman, Jared G. Galloway, Ariella L. Gladstein, Gregor Gorjanc, Bing Guo, Ben Jeffery, Warren W. Kretzschumar, Konrad Lohse, Michael Matschiner, Dominic Nelson, Nathaniel S. Pope, Consuelo D. Rasmussen, Murillo F. Raza, Shahina Rizvi, Guillermo Sellés, Matthias Steinrücken, Jonathan Terhorst, Kevin R. Thornton, Hugo van Kemenade, Anthony W. Wohns, Yan Wong, Simon Gravel, Andrew D. Kern, Jere Koskela, Peter L. Ralph, and Jerome Kelleher. Efficient ancestry and mutation simulation with msprime 1.0. *Genetics*, 220(3):iyab229, 2022. doi: 10.1093/genetics/iyab229.

Ryan N. Gutenkunst, Ryan D. Hernandez, Scott H. Williamson, and Carlos D. Bustamante. Inferring the joint demographic history of multiple populations from multidimensional SNP frequency data. *PLoS Genetics*, 5(10):e1000695, 2009. doi: 10.1371/journal.pgen.1000695.

Jerome Kelleher, Yan Wong, Anthony W. Wohns, Chaimaa Fadil, Patrick K. Albers, and Gil McVean. Inferring whole-genome histories in large population samples. *Nature Genetics*, 51(9):1330–1338, 2019. doi: 10.1038/s41588-019-0483-y.

Heng Li and Richard Durbin. Inference of human population history from individual whole-genome sequences. *Nature*, 475(7357):493–496, 2011. doi: 10.1038/nature10231.

Anna-Sapfo Malaspinas, Michael C. Westaway, Craig Muller, et al. A genomic history of Aboriginal Australia. *Nature*, 538:207–214, 2016. doi: 10.1038/nature18299.

Pier Francesco Palamara, Todd Lencz, Ariel Darvasi, and Itsik Pe'er. Length distributions of identity by descent reveal fine-scale demographic history. *American Journal of Human Genetics*, 91:809–822, 2012. doi: 10.1016/j.ajhg.2012.08.030.

Peter Ralph and Graham Coop. The geography of recent genetic ancestry across Europe. *PLoS Biology*, 11: e1001555, 2013. doi: 10.1371/journal.pbio.1001555.

Stephan Schiffels and Richard Durbin. Inferring human population size and separation history from multiple genome sequences. *Nature Genetics*, 46(8):919–925, 2014. doi: 10.1038/ng.3015.

Regev Schweiger and Richard Durbin. Ultra-fast genome-wide inference of pairwise coalescence times. *Genome Research*, 33(7):1023–1031, 2023. doi: 10.1101/gr.277665.123.

Leo Speidel, Marie Forest, Sinan Shi, and Simon R. Myers. A method for genome-wide genealogy estimation for thousands of samples. *Nature Genetics*, 51(9):1321–1329, 2019. doi: 10.1038/s41588-019-0484-x.

Brian C. Zhang, Arjun Biddanda, Árni Freyr Gunnarsson, Fergus Cooper, and Pier Francesco Palamara. Biobank-scale inference of ancestral recombination graphs. *Nature Genetics*, 55(5):768–776, 2023. doi: 10.1038/s41588-023-01379-x.

Supporting information

S1 Text. Detailed mathematical derivations and implementation notes. Contains the full Gamma-SMC emission and transition equations, flow field construction algorithm, bilinear interpolation details, numerical precision analysis, and extended benchmarking results for additional demographic scenarios.